

UNIVERSITÉ DE CAEN NORMANDIE
Licence Informatique – 3^{ème} année (L3)
Matière : Projet 2

Dota 2 Trajectory Analyzer

Fouille de Motifs Spatio-Temporels et Clustering de Trajectoires
E-sport pour en extraire les stratégies

Rapport de Projet de Conception

Équipe de développement :

Elias OKAT
Uzay TURKMENEL
Paul AUBERT

Année universitaire 2025-2026

Table des matières

1	Introduction	3
1.1	Description Générale du Projet	3
1.2	Plan du Rapport	3
2	Objectifs du Projet	3
2.1	Contexte Métier : Le Jeu DotA 2	3
2.2	La Problématique de l'Analyse de Trajectoires	3
2.3	Points-Clés et Grandes Étapes	3
2.4	Travaux Existants et Positionnement	4
3	Fonctionnalités Implémentées	4
3.1	Description des Fonctionnalités	4
3.2	Qualité Logicielle	5
3.3	Organisation du Projet : Répartition des Tâches	5
4	Éléments Techniques	5
4.1	Choix Technologique : Python Scientifique	5
4.2	Description des Données	6
4.3	Description des Algorithmes : Pseudo-code MDL Compress	6
4.4	Structures de Données	7
5	Architecture du Projet	7
5.1	Paquetages Non Standards et Dépendances	7
5.2	Diagrammes des Modules et des Classes	8
5.3	Chaîne de Traitement	8
6	Expérimentations	8
6.1	Cas d'Utilisation	8
6.2	Validation de la Compression par Seuil (MDL)	9
6.2.1	Vérification du Fonctionnement	10
6.2.2	Invariance au Bruit et Stabilité Structurale	10
6.2.3	Validation Visuelle de la Robustesse	11
6.2.4	Évaluation Comparative avec les Méthodes Standards (Baseline)	11
6.2.5	Évaluation de la Complexité Computationnelle	13
6.2.6	Analyse de Sensibilité : Justification du paramètre w_{error}	13
6.2.7	Robustesse au Bruit Gaussien	14
6.3	Évaluation des Algorithmes de Clustering	15
6.3.1	Optimisation de k : Compromis entre Topologie et Sémantique	15
6.3.2	Évaluation Topologique Comparative des Algorithmes	16
6.3.3	Analyse de Sensibilité et Robustesse de la Propagation d'Affinité (AP)	17
6.3.4	Validation Croisée et Stabilité Bootstrap de l'AP	17
6.3.5	Scalabilité vs Qualité Sémantique : Le Choix Final	19
6.4	Fouille de Motifs Séquentiels et Interprétation Stratégique	20
6.4.1	Gestion de la Volumétrie et Malédiction des Séquences Longues	20
6.4.2	Analyse des Motifs Séquentiels (PrefixSpan) et Graphe de Markov	22
6.5	Récapitulatif des Hyperparamètres du Pipeline	24

7	Conclusion	25
7.1	Récapitulatif de la Problématique et de la Réalisation	25
7.2	Récapitulatif des Résultats	25
7.3	Reproductibilité des Expériences	25
7.4	Propositions d'Améliorations	26

1 Introduction

1.1 Description Générale du Projet

L'avènement de l'e-sport (sport électronique) a généré des volumes de données sans précédent. Dans des jeux compétitifs complexes comme *DotA 2* (Defense of the Ancients 2), chaque partie génère des dizaines de milliers de points de données spatio-temporelles, reflétant les moindres clics et déplacements des joueurs. Si ces données brutes sont massivement stockées, leur exploitation stratégique reste un défi majeur pour les analystes et les équipes professionnelles.

Ce projet s'inscrit dans cette dynamique d'analyse automatique de données de jeu appliquée à l'e-sport. Notre objectif est de concevoir et d'implémenter une chaîne de traitement algorithmique capable d'ingérer les trajectoires brutes des joueurs pour en extraire des régularités de mouvement et des stratégies partielles interprétables.

1.2 Plan du Rapport

Ce rapport détaillera l'intégralité de notre démarche. Après avoir exposé les objectifs et la problématique du projet (Section 2), nous décrirons les fonctionnalités implémentées et l'organisation de l'équipe (Section 3). Nous présenterons ensuite les éléments techniques et algorithmiques (Section 4) ainsi que l'architecture logicielle du système (Section 5). Nous consacrerons une large partie à l'évaluation expérimentale de nos modèles — de la compression géométrique au clustering spatial, jusqu'à l'extraction finale des motifs stratégiques (Section 6). Enfin, nous conclurons sur les apports de ce projet et ses perspectives d'évolution (Section 7).

2 Objectifs du Projet

2.1 Contexte Métier : Le Jeu DotA 2

DotA 2 est un jeu vidéo de type MOBA (Multiplayer Online Battle Arena). Deux équipes de cinq joueurs s'affrontent sur une carte fermée et asymétrique, composée de trois voies principales (les *Lanes*), séparées par une rivière et des zones de forêts (la *Jungle*). L'objectif est de détruire la base adverse. La victoire repose non seulement sur les réflexes mécaniques (micro-gestion), mais surtout sur la prise de décision globale, les déplacements coordonnés et le contrôle de la carte (macro-gestion).

2.2 La Problématique de l'Analyse de Trajectoires

La donnée brute dont nous disposons est un flux continu de coordonnées (X, Y) enregistrées à intervalles réguliers (les *ticks*). Cette donnée pose trois problèmes majeurs :

1. **Le volume** : Une partie génère des millions de points.
2. **Le bruit stochastique** : Les joueurs cliquent frénétiquement pour maintenir leur personnage en mouvement (micro-tremblements), générant un nuage de points erratique qui ne représente aucune réelle intention de déplacement.
3. **L'absence de sémantique** : Un point $(120, 115)$ n'a aucun sens stratégique pour une machine.

L'objectif fondamental de ce projet est donc de répondre à la question scientifique suivante : **Comment transformer de manière non supervisée un volume massif de micro-clics erratiques en un alphabet de comportements locaux (ganks, farming, rotations) compréhensible par un analyste humain ?**

2.3 Points-Clés et Grandes Étapes

Le cœur mathématique du projet repose sur un pipeline de données séquentiel en trois phases :

1. **Phase 1 - Compression (MDL)** : Nettoyage des nuages de points erratiques pour extraire des segments de droites directionnels.
2. **Phase 2 - Discrétisation Spatiale (TRACCLUS + AP)** : Calcul de la matrice de similarité entre les segments, puis clustering pour découvrir les "Macro-Zones" de la carte.
3. **Phase 3 - Extraction Sémantique (PrefixSpan)** : Recodage des trajectoires en séquences d'états (via Run-Length Encoding) et fouille de motifs fréquents pour identifier les véritables stratégies (ex : zone A $\rightarrow$ zone B $\rightarrow$ zone C).

2.4 Travaux Existants et Positionnement

L'analyse des déplacements dans les jeux vidéo compétitifs a fait l'objet de plusieurs approches académiques dans le domaine du *Game Analytics* [4]. Pour extraire de l'information stratégique à partir de coordonnées spatiales, la littérature s'appuie généralement sur trois grandes méthodologies, qui présentent cependant certaines limites face à la complexité de *DotA 2* :

1. **La cartographie statistique statique (Heatmaps)** : L'approche la plus répandue dans l'industrie consiste à agréger les positions des joueurs pour générer des cartes de chaleur. Si cette méthode permet d'identifier visuellement les zones de forte activité, elle écrase totalement la dimension temporelle et séquentielle. Une *heatmap* montre qu'un joueur a été dans la jungle puis au centre, mais ne permet pas de savoir s'il s'agit d'une rotation offensive ou d'un repli.
2. **L'analyse séquentielle sur grilles prédéfinies** : Pour réintégrer le temps, de nombreux travaux d'analyse de trajectoires standards [3] découpent arbitrairement la carte en une grille mathématique ou utilisent les zones sémantiques codées en dur dans le jeu. Les trajectoires sont alors converties en séquences de zones. La limite majeure de cette approche est le **biais humain** : le découpage manuel ne correspond pas nécessairement aux véritables flux de déplacement des joueurs, générant du bruit aux frontières arbitraires.
3. **Le clustering direct par Dynamic Time Warping (DTW)** : D'autres études tentent de comparer directement les trajectoires brutes en utilisant des métriques de distance temporelles flexibles comme le DTW ou la distance de Fréchet. Bien que mathématiquement robustes, ces méthodes se heurtent au mur de la scalabilité ($\mathcal{O}(N^2)$ voire $\mathcal{O}(N^3)$) sur des millions de points et sont extrêmement sensibles aux micro-tremblements de la souris.

Positionnement de notre étude : Notre projet se démarque de ces approches par la conception d'un pipeline **hybride et totalement guidé par la donnée (*data-driven*)**. Pour contourner les limites du DTW et de l'échantillonnage classique, nous nous appuyons sur le paradigme *Partition-and-Group* introduit par Lee et al. [1]. Nous supprimons le bruit en amont via leur algorithme de compression géométrique **MDL**. Ensuite, contrairement au découpage arbitraire en grille, notre zonage est découvert dynamiquement de manière non supervisée par l'algorithme **Affinity Propagation** combiné à la distance experte **TRACCLUS** [1]. Enfin, l'application de l'algorithme **PrefixSpan**, formalisé par Pei et al. [2], sur cet alphabet purifié permet de retrouver la dynamique temporelle perdue par les simples *Heatmaps*.

3 Fonctionnalités Implémentées

Le projet a été développé pour être utilisable à la fois par des ingénieurs (via une CLI offrant 11 commandes d'automatisation et de batching) et par des analystes finaux via une Interface Graphique (GUI) avancée (v1.1.0).

3.1 Description des Fonctionnalités

Conçue avec le framework *CustomTkinter* selon le patron de conception MVC (Modèle-Vue-Contrôleur), l'application propose un environnement ergonomique structuré autour de 7 modules interactifs :

- **Accueil (Dashboard)** : Un tableau de bord dynamique présentant les métriques clés de la base de données en temps réel (Matches ingérés, Matches compressés, Clusters générés, Motifs fréquents extraits).
- **Overlay Carte et Exploration spatiale** : Outil de visualisation superposant les trajectoires sur la carte officielle de *DotA 2*. Il intègre un filtrage granulaire et un curseur temporel permettant d'inspecter les déplacements *tick par tick*.
- **Module de Compression (MDL)** : Interface de traitement par lots intégrant un suivi en temps réel via une console de *logs* (taux de réduction, empreinte mémoire).
- **Atelier de Clustering** : Espace d'exploration permettant de paramétrer les algorithmes spatiaux. L'interface s'adapte dynamiquement à l'algorithme choisi (ex : sélection du *damping* pour l'AP) et affiche le pavage spatial généré.
- **Comparateur Synchronisé (Brut vs Compressé)** : Module d'inspection visuelle affichant deux cartes côte à côte. Le système pré-calcule l'intégralité des *frames* en mémoire pour garantir une lecture temporelle parfaitement fluide et synchronisée.
- **Fouille Séquentielle (PrefixSpan)** : Interface dédiée à l'extraction sémantique restituant les résultats sous forme de tableau textuel filtrable, d'histogramme des fréquences, et de graphe de transitions interactif.
- **Orchestrateur End-to-End (Pipeline)** : Module automatisant la chaîne de valeur. L'exécution parallèle s'appuie sur le *multiprocessing* natif (CPU_count - 2 cœurs). Des indicateurs visuels suivent l'avancement des étapes jusqu'à la génération des résultats analytiques finaux.

3.2 Qualité Logicielle

Le projet est accompagné d'une suite de **72 tests unitaires** couvrant l'ensemble des modules critiques (compression, géométrie, clustering, fouille de motifs). Ces tests sont exécutables via **pytest** et permettent de vérifier systématiquement le bon fonctionnement du pipeline après chaque modification du code.

3.3 Organisation du Projet : Répartition des Tâches

Le développement de ce projet a nécessité une répartition des rôles au sein de notre groupe de trois étudiants :

- **Elias OKAT** : Parsing des fichiers CSV bruts avec Pandas, implémentation de l'algorithme de compression MDL, K-Médoïdes pour le clustering et implémentation des benchmarks pour MDL et clustering.
- **Uzay TURKMENEL** : Calculs géométriques vectorisés (distances TRACCLUS) et développement des algorithmes de clustering spatiaux.
- **Paul AUBERT** : Recodage RLE, développement *from scratch* de l'algorithme PrefixSpan et modélisation sous forme de Graphe de Markov.

4 Éléments Techniques

4.1 Choix Technologique : Python Scientifique

L'ensemble du projet est développé en **Python 3**, un choix motivé par la maturité de son écosystème pour le calcul scientifique et l'analyse de données. Trois bibliothèques constituent le socle technique de notre pipeline :

- **NumPy** : toutes les opérations critiques (distances TRACCLUS, matrices de similarité $N \times N$, compression MDL) sont vectorisées via `numpy.ndarray`, ce qui permet des gains de performance de l'ordre de $\times 100$ par rapport à des boucles Python classiques.

- **Pandas** : utilisé pour l'ingestion et le nettoyage des fichiers CSV bruts, chaque fichier contenant un *tick* par ligne avec les coordonnées des 10 joueurs.
- **Matplotlib** : génération de l'ensemble des visualisations du projet, des *overlays* de trajectoires sur la carte de jeu aux graphiques multi-panneaux des benchmarks expérimentaux.

Les algorithmes centraux du pipeline, notamment l'*Affinity Propagation* et *PrefixSpan*, ont été implémentés intégralement par nos soins (*from scratch*) sans recours à des bibliothèques externes, afin de maîtriser la complexité algorithmique et d'adapter finement chaque étape à notre contexte de données e-sport.

4.2 Description des Données

Nos expérimentations portent sur un corpus de 70 matchs réels de *DotA 2* (fichiers CSV), représentant 10 joueurs par match. Chaque fichier contient un *tick* par ligne avec les coordonnées (X, Y) des 10 joueurs. Après compression avec $w_{error} = 12$, ce corpus génère environ 18 000 segments. Pour l'étape de fouille de motifs (Section 6.4), un sous-ensemble de 30 matchs est utilisé, produisant 1 174 séquences fréquentes avec un support minimum de 15%.

Pour illustrer la transformation opérée par notre pipeline, la Figure 1 présente la transition entre la donnée brute et la donnée structurée :

[illegible]

(a) Extrait du flux brut (CSV).

```
{
  "match_id": "3841665963",
  "w_error": 12.0,
  "compression_info": {
    "total_players": 10,
    "timestamp": "2020-03-25T22:58:39.620842",
    "algorithm": "MDL Greedy",
    "formula": " $Cost = L(H) + w\_error * L(D|H)$ "
  },
  "players": [
    {
      "player_id": 0,
      "num_segments": 75,
      "num_original_points": 0,
      "compression_rate": 0.0,
      "segments": [
        {
          "start": {
            "x": 80.0,
            "y": 92.0,
            "tick": 28320
          },
          "end": {
            "x": 100.0,
            "y": 168.0,
            "tick": 29310
          },
          "length": 78.587530817554,
          "angle": 75.25643716352927
        }
      ]
    }
  ]
}
```

(b) Séquence encodée après traitement.

FIGURE 1 – Transition entre la donnée brute et la donnée structurée par le pipeline.

4.3 Description des Algorithmes : Pseudo-code MDL Compress

L’algorithme glouton (Greedy) inspiré du principe du Minimum Description Length (MDL) [8], tel qu’adapté par Lee et al. [1] pour la simplification de trajectoires, est la clé de voûte du débruitage. Son implémentation vectorisée en NumPy permet d’accélérer significativement le traitement.

Note terminologique : Notre implémentation s’inspire du critère MDL tel que décrit par Lee et al. [1], où le coût d’encodage est simplifié sous la forme d’un seuil de distance perpendiculaire (w_{error}). Il ne s’agit pas d’une implémentation complète du principe MDL au sens de Rissanen [8] (minimisation de $L(H) + L(D|H)$), mais d’une approximation pratique qui conserve l’idée de compromis entre concision et fidélité.

Algorithme 1 : CompresserTrajectoireMDL

```
Input :  $T = [p_1, p_2, \dots, p_n]$  (liste ordonnée des coordonnées du joueur)
           $w_{error}$  (seuil de tolérance de distance perpendiculaire max)
Output :  $S$  (liste des segments compressés)
 $S \leftarrow \emptyset$ 
 $debut \leftarrow 1$ 
while  $debut < n$  do
     $meilleur\_point\_fin \leftarrow debut + 1$ 
    for  $fin \leftarrow debut + 2$  to  $n$  do
         $points\_intermediaires \leftarrow T[debut + 1 \dots fin - 1]$ 
         $distances \leftarrow \text{DistancePerpendiculaire}(points\_intermediaires, \text{ligne}(T[debut], T[fin]))$ 
        if  $\max(distances) < w_{error}$  then
             $meilleur\_point\_fin \leftarrow fin$  // L'extension du segment est valide
        else
            break // Tolérance brisée, le segment s'arrête
     $S.\text{ajouter}(\text{Segment}(T[debut], T[meilleur\_point\_fin]))$ 
     $debut \leftarrow meilleur\_point\_fin$  // Reprise au dernier point validé
return  $S$ 
```

4.4 Structures de Données

L'ingestion de millions de points a nécessité une conception rigoureuse des structures de données pour éviter la saturation de la RAM :

- **Modélisation objet bas-niveau** : Les points bruts et les segments sont encapsulés dans des `@dataclass` Python (`TrajectoryPoint`, `Segment`) proposant des méthodes vectorielles `numpy.ndarray` pour les calculs angulaires instantanés.
- **Matrice TRACCLUS** : Les distances inter-segments ($d_{\perp}$, $d_{\parallel}$, d_{θ}) sont stockées dans une matrice carrée symétrique `numpy.ndarray` de dimension $N \times N$ typée en `float32`. Sa diagonale est initialisée à la médiane des similarités avant injection dans l’Affinity Propagation.
- **Séquences RLE et SPMF** : Après compression, les séquences de chaque joueur sont converties au format standard d’exploration SPMF (ex : 5 -1 3 -1 5 -1 -2), où -1 sépare les itemsets et -2 termine la séquence, stockées dans un dictionnaire associant la séquence à son support.

5 Architecture du Projet

Afin de répondre à notre problématique, nous avons conçu une application modulaire et robuste, séparant strictement la logique algorithmique de l’interface utilisateur.

5.1 Paquetages Non Standards et Dépendances

Pour manipuler la volumétrie des données, nous nous sommes appuyés sur **Pandas** pour le chargement des CSV initiaux (un *tick* et les coordonnées (X, Y) des 10 joueurs par ligne), **NumPy** pour les calculs vectorisés, **Matplotlib** pour la génération des *overlays* géographiques, et **NetworkX** pour le rendu du graphe de Markov.

Point technique majeur : Afin de maîtriser totalement la complexité algorithmique, l’algorithme de *Clustering Affinity Propagation* [5] (message-passing) et l’algorithme de fouille de motifs *PrefixSpan* (pattern-growth) **ne proviennent d’aucune bibliothèque externe**. Ils ont été intégralement implémentés et optimisés par nos soins (*from scratch*) en Python pur.

5.2 Diagrammes des Modules et des Classes

Notre dépôt est structuré autour de trois grands packages, illustrés par l'architecture globale du système détaillée dans la Figure 2 :

- `dota_analytics/` : La bibliothèque cœur contenant la logique mathématique (`geometry.py`, `compression.py`, `clustering.py`, `mining.py`).
- `mvc/` : Les éléments de l'interface graphique répartis en `models`, `controllers`, et `views`.
- `benchmark/` : Un dossier regroupant les scripts d'expérimentation isolés (`exp0_werror.py`, `exp2_prefixspan`, etc.) utilisés pour la rédaction de ce rapport.

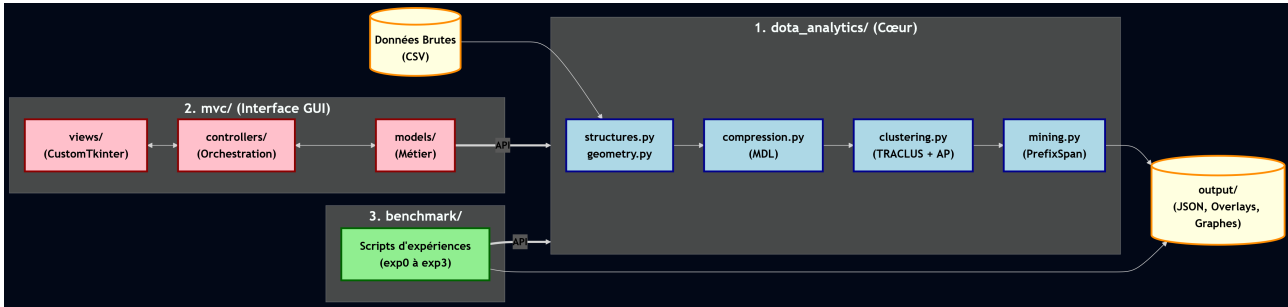


FIGURE 2 – Architecture logicielle et flux de données du Dota 2 Trajectory Analyzer.

5.3 Chaîne de Traitement

Le projet s'organise selon un patron MVC : la bibliothèque `dota_analytics` centralise les modèles et traitements réutilisables, tandis que le dossier `benchmark` fait office de laboratoire d'expérimentation, permettant de tester, calibrer et comparer chaque algorithme de manière isolée avant son intégration dans le pipeline final. Les données transitent séquentiellement au travers du pipeline décrit en Section 2.3 : les CSV bruts sont d'abord ingérés et compressés par `compression.py`, les segments résultants sont comparés via `metrics.py` (distance TRACCLUS) et groupés par `clustering.py`, puis recodés par `recoding.py` et analysés par `mining.py`.

6 Expérimentations

Cette section présente l'ensemble de nos expérimentations, des cas d'utilisation concrets aux résultats quantitatifs détaillés.

6.1 Cas d'Utilisation

Afin d'illustrer concrètement les fonctionnalités décrites en Section 3, voici deux cas d'utilisation représentatifs de notre application.

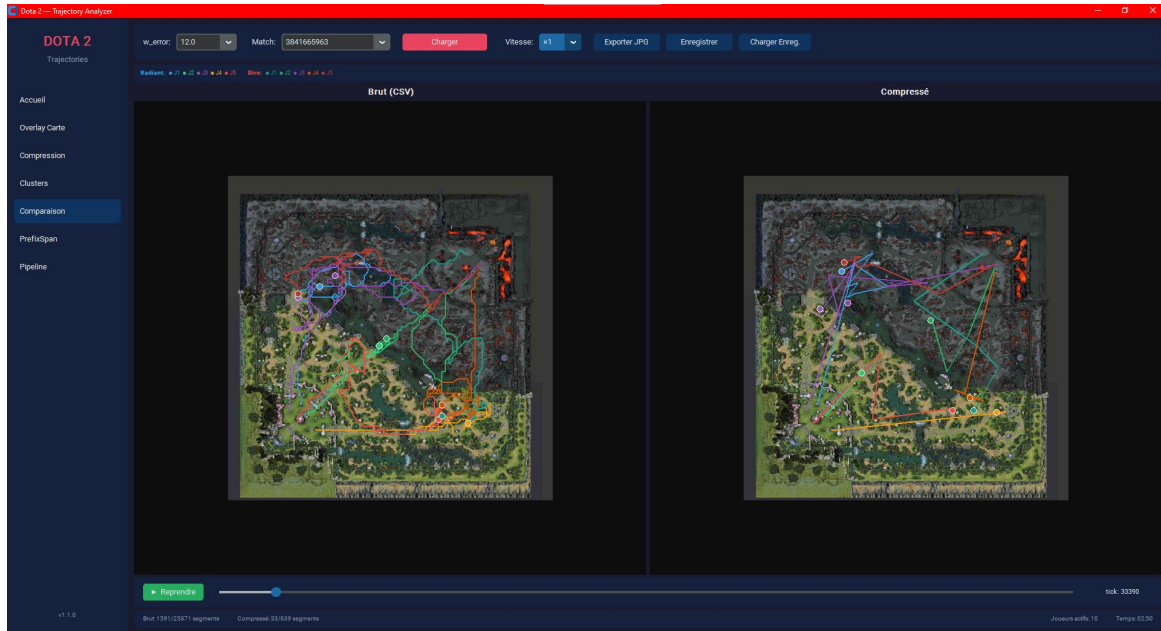


FIGURE 3 – Cas d'utilisation : Module de comparaison synchronisée Brut vs Compressé.

La Figure 3 montre le module de comparaison synchronisée, qui affiche deux cartes côte à côte permettant d'inspecter visuellement l'effet de la compression MDL sur une trajectoire.

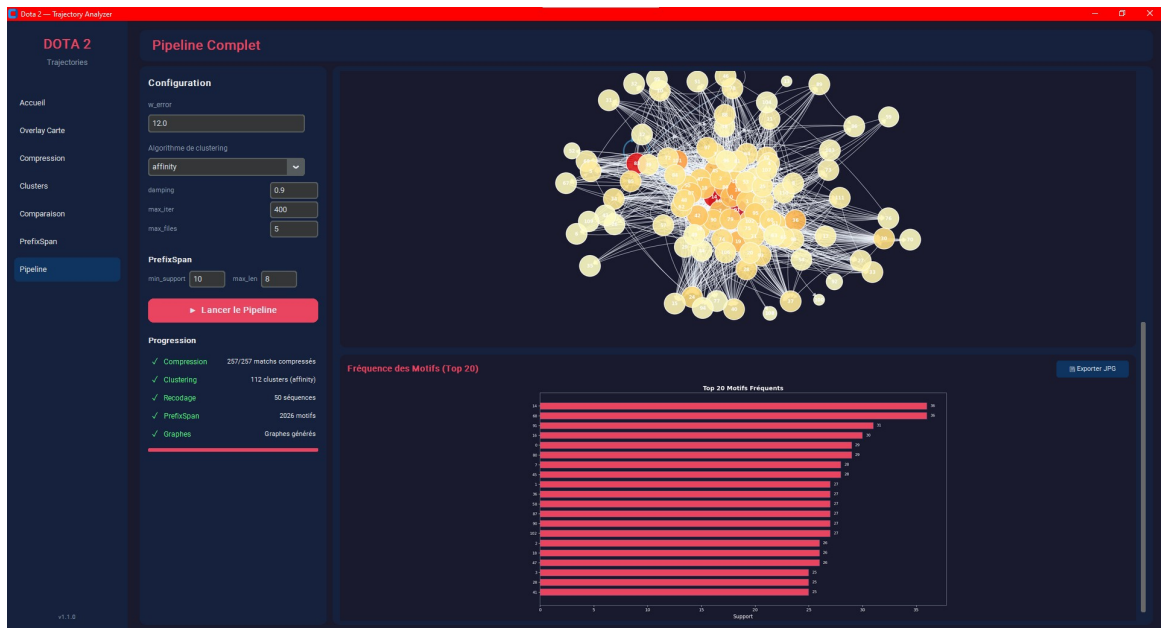


FIGURE 4 – Cas d'utilisation : Exécution de l'orchestrateur de Pipeline avec restitution de l'analyse PrefixSpan.

La Figure 4 illustre l'orchestrateur end-to-end, qui automatise l'enchaînement complet du pipeline et restitue les résultats de l'analyse PrefixSpan.

6.2 Validation de la Compression par Seuil (MDL)

Afin de valider rigoureusement notre implémentation de l'algorithme MDL, il était indispensable de dépasser la simple vérification visuelle pour adopter une démarche d'évaluation objective et quantitative.

6.2.1 Vérification du Fonctionnement

Pour évaluer la qualité de la compression sur un large volume de données, nous avons mesuré l'erreur spatiale (RMSE) et le taux de compression sur un ensemble représentatif de trajectoires.

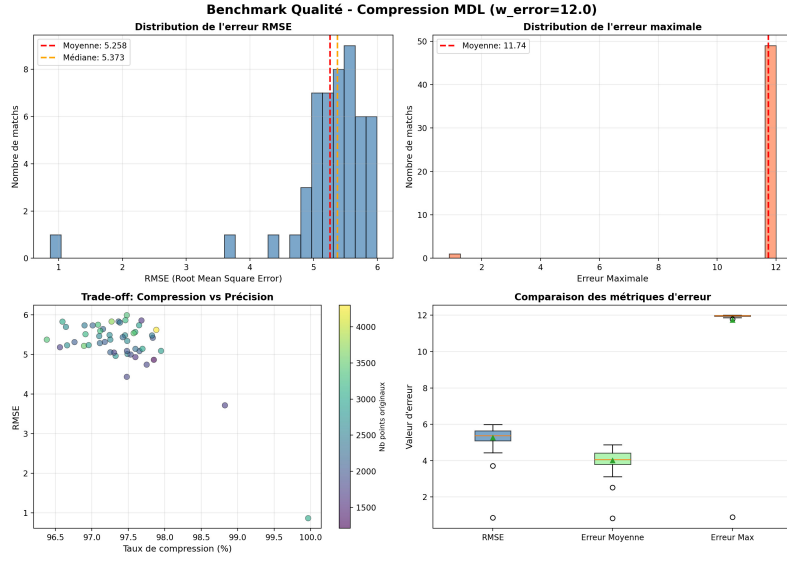


FIGURE 5 – Distribution de l'erreur maximale et compromis Compression/Précision.

L'analyse de la Figure 5 met en évidence deux observations importantes :

- **Plafonnement de l'Erreur Maximale** : La distribution montre que l'erreur maximale est strictement contenue juste sous la barre des 12 unités. Cela confirme que l'approche "Greedy" respecte notre paramètre $w_{error} = 12$.
- **Trade-off Compression vs Précision** : On observe une concentration optimale entre 96,5% et 98,0% de taux de compression, pour un RMSE contenu.

6.2.2 Invariance au Bruit et Stabilité Structurale

Pour tester la robustesse face aux comportements erratiques, nous avons injecté un bruit gaussien artificiel sur les trajectoires.

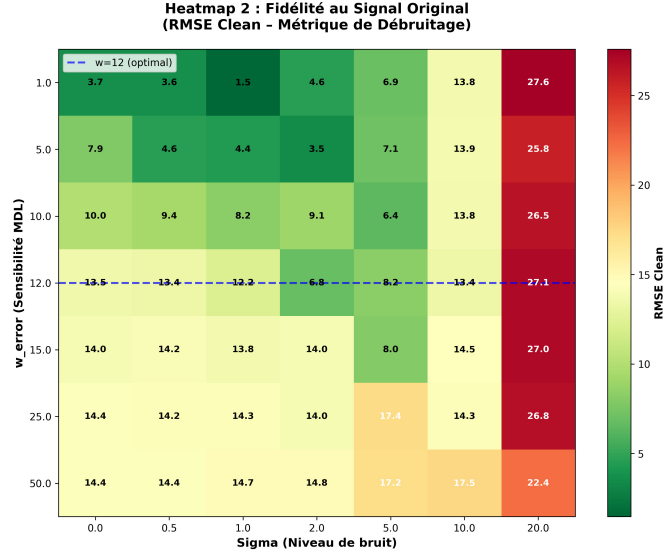


FIGURE 6 – Étude de stabilité structurelle face au bruit gaussien : Évolution du nombre de segments et du RMSE.

Comme illustré par la Figure 6, l'étude révèle que pour $w_{error} = 12.0$, le nombre de segments générés reste parfaitement stable pour des niveaux de bruit $\sigma \leq 2.0$. L'algorithme absorbe efficacement les petites perturbations sans modifier la forme globale de la trajectoire.

6.2.3 Validation Visuelle de la Robustesse

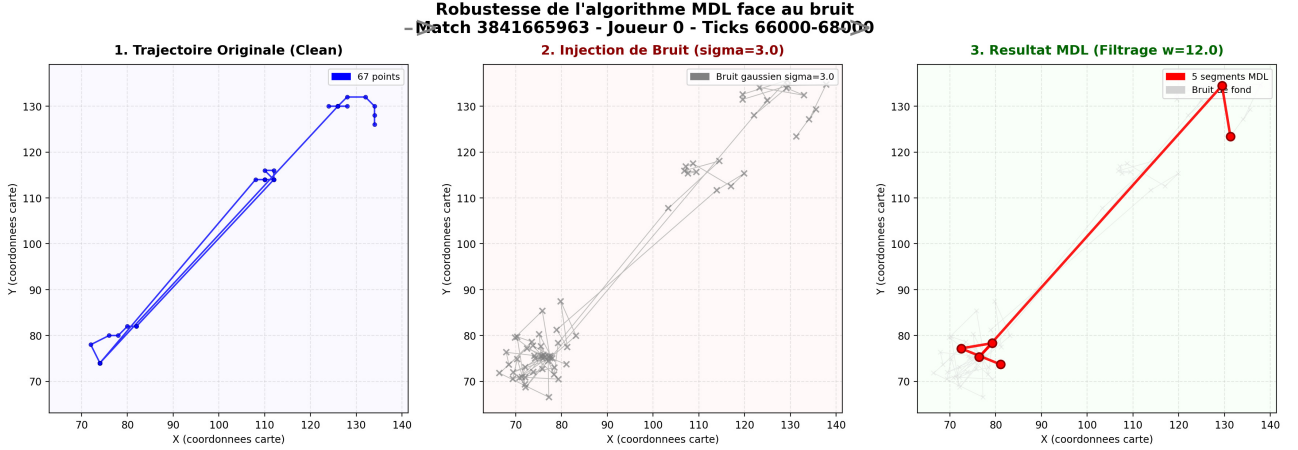


FIGURE 7 – Reconstruction d'une trajectoire à partir de données hautement bruitées.

Comme le démontre la Figure 7, face à un nuage chaotique simulant de multiples micro-clics, la compression MDL parvient à extraire des segments nets passant par le centre du nuage de points, conservant ainsi la forme globale du déplacement.

6.2.4 Évaluation Comparative avec les Méthodes Standards (Baseline)

Nous comparons MDL à l'échantillonnage uniforme et à l'algorithme de Douglas-Peucker [6] à **taux de compression égal** ($\approx 95\%$), en ajustant les paramètres de chaque méthode pour atteindre ce taux.

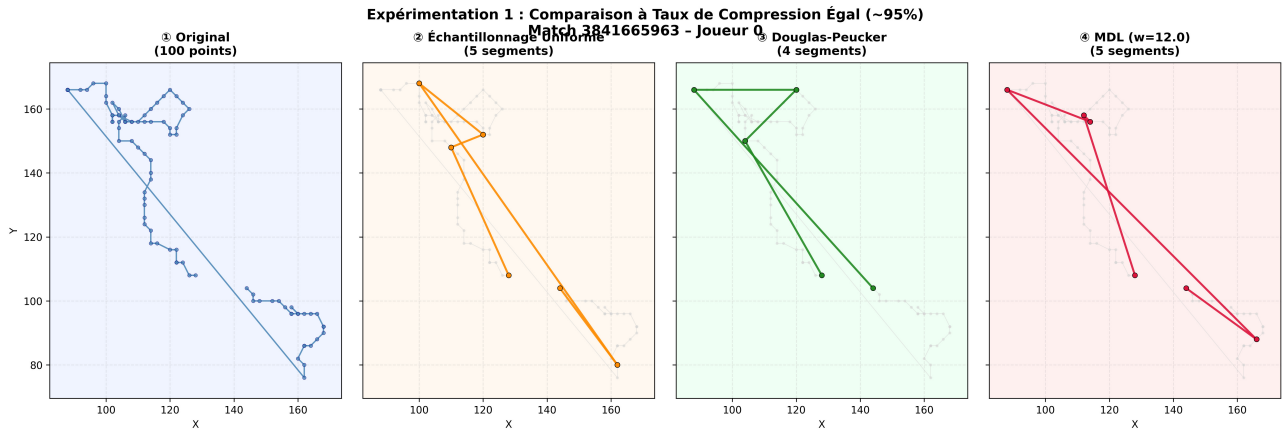


FIGURE 8 – Comparaison visuelle des algorithmes de compression face à une trajectoire réelle.

Sur la Figure 8, on observe visuellement que l'échantillonnage Uniforme crée des trajectoires erratiques, tandis que Douglas-Peucker se laisse piéger par les *outliers*.

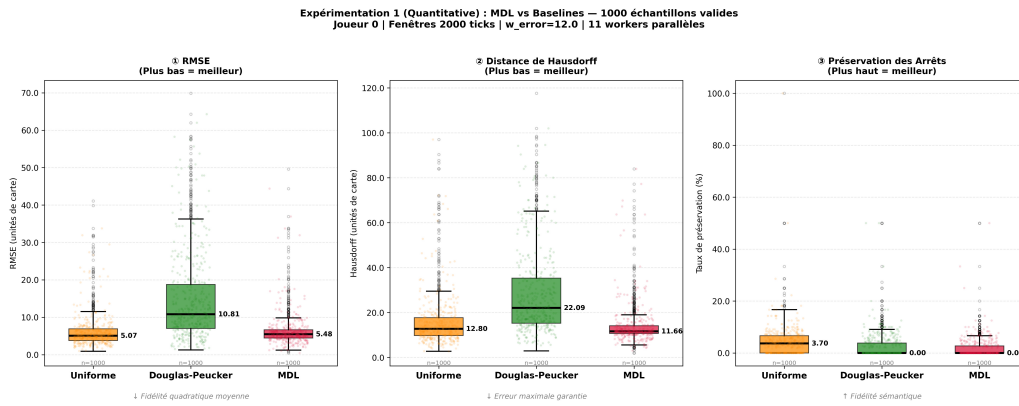


FIGURE 9 – Analyse statistique (Boxplots) des algorithmes sur 1000 trajectoires de joueurs issues des matchs disponibles.

L'analyse statistique détaillée (Figure 9) révèle les différences suivantes :

- **Fidélité Quadratique Moyenne (RMSE) :** L'échantillonnage uniforme (médiane à 5.07) et MDL (5.48) obtiennent des résultats nettement meilleurs que Douglas-Peucker (10.81).
- **Erreur Maximale (Distance de Hausdorff) :** MDL obtient la meilleure fidélité géométrique (médiane à 11.66, contre 12.80 pour l'uniforme et 22.09 pour DP), ce qui limite l'écart maximal entre la trajectoire originale et la trajectoire compressée.
- **Information temporelle :** Ces trois méthodes, étant purement spatiales, ne préservent pas l'information de durée de pause aux points intermédiaires supprimés.

Le Tableau 1 résume ces résultats de manière synthétique.

TABLE 1 – Récapitulatif de la comparaison des algorithmes de compression (à taux de compression $\approx 95\%$).

Métrique	Uniforme	Douglas-Peucker	MDL
RMSE (médiane)	5.07	10.81	5.48
Hausdorff (médiane)	12.80	22.09	11.66
Préservation temporelle	Non	Non	Non

6.2.5 Évaluation de la Complexité Computationnelle

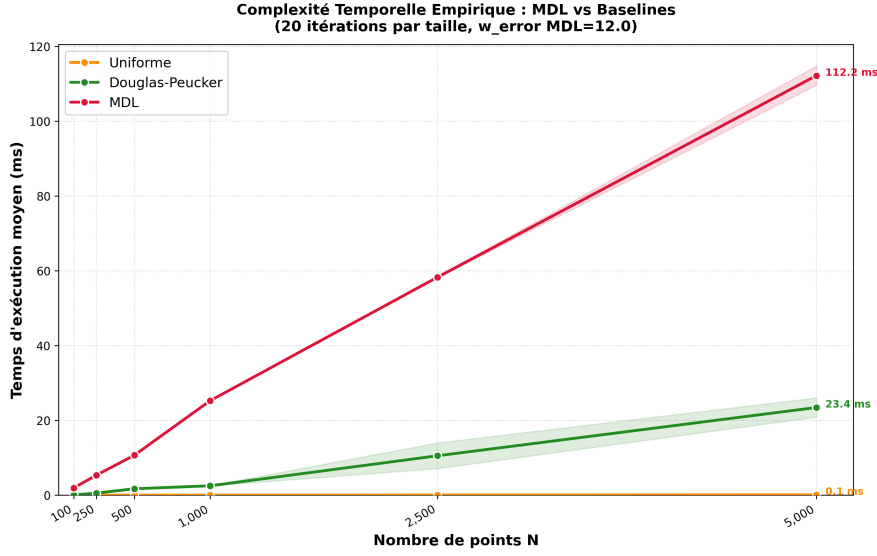


FIGURE 10 – Benchmark de complexité temporelle empirique ($w_{error} = 12.0$).

D'après la Figure 10, l'implémentation MDL affiche un coût computationnel de l'ordre de $\mathcal{O}(N^2)$ dans le pire cas, atteignant 7,19 secondes pour 5000 points. En pratique, le *break* anticipé dans la boucle interne réduit ce coût moyen, ce qui explique la courbe empirique située entre $\mathcal{O}(N^{1.5})$ et $\mathcal{O}(N^2)$. Un temps de traitement *offline* de 7 secondes par demi-heure de jeu reste acceptable pour un usage non temps-réel.

6.2.6 Analyse de Sensibilité : Justification du paramètre w_{error}

Pour justifier empiriquement le choix de notre tolérance d'erreur, nous avons fait varier w_{error} de 0 à 50 et mesuré son impact sur l'ensemble de la chaîne de traitement.

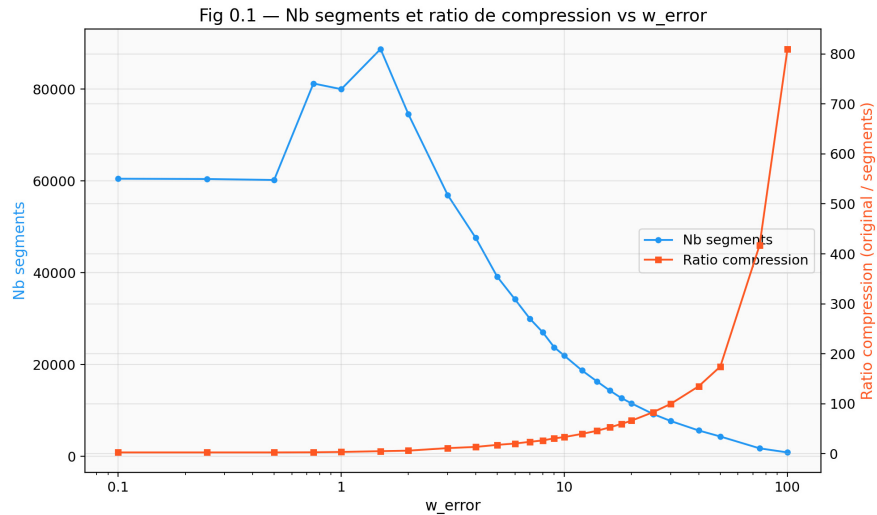
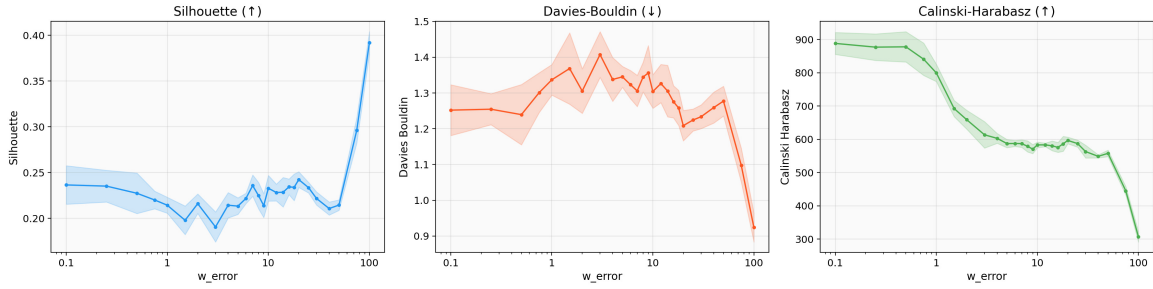
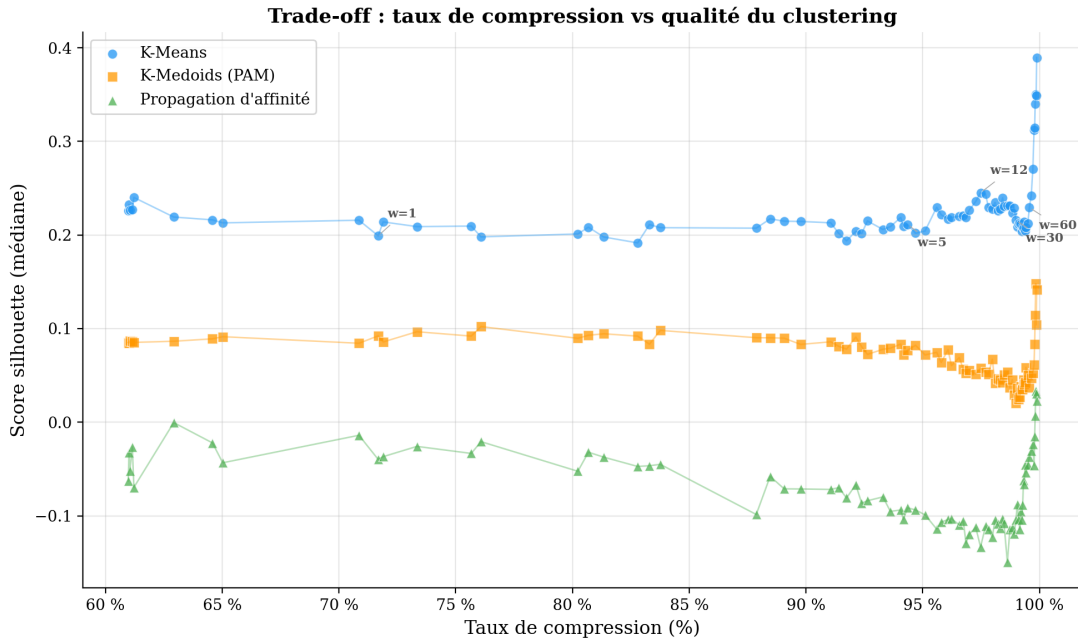


FIGURE 11 – Évolution du nombre de segments et du ratio de compression en fonction de w_{error} .

La Figure 11 montre qu'au-delà de $w_{error} = 12$, le gain en compression marginal plafonne autour de 96% et ne justifie plus la perte d'information géométrique.

Fig 0.4 — Métriques de clustering vs w_{error} FIGURE 12 – Évolution de la qualité topologique du clustering en fonction de w_{error} .

Pour confirmer ce choix purement spatial, la Figure 12 illustre l’impact de la compression sur la qualité future du clustering. L’observation conjointe montre que le score de cohésion (Silhouette [7]) présente une forte variance pour les petites valeurs de w_{error} avant de se stabiliser autour de $w_{error} \approx 12$. Ce paramètre offre donc un bon compromis entre précision géométrique et qualité du clustering en aval.

FIGURE 13 – Trade-off entre taux de compression et qualité du clustering (score silhouette médian) pour trois algorithmes. Le paramètre $w_{error} = 12$ est annoté à $\sim 96\%$ de compression.

La Figure 13 synthétise ce compromis en traçant directement le score silhouette en fonction du taux de compression pour les trois algorithmes de clustering. On constate que K-Means maintient un score stable (~ 0.21) sur toute la plage de compression, tandis que K-Médoïdes et Affinity Propagation sont plus sensibles aux compressions extrêmes ($> 98\%$). Au point $w_{error} = 12$ ($\sim 96\%$ de compression), les trois méthodes se situent dans leur zone de stabilité, confirmant que notre choix de paramètre ne dégrade pas la qualité du clustering en aval.

6.2.7 Robustesse au Bruit Gaussien

Afin de valider la capacité de l’algorithme MDL à filtrer le bruit intrinsèque aux données de clics, nous avons mené une expérience de robustesse par injection de bruit gaussien $\mathcal{N}(0, \sigma^2)$ avec σ croissant de 0 à 20.

Robustesse MDL au Bruit Gaussien ($w_{\text{error}} = 12$)

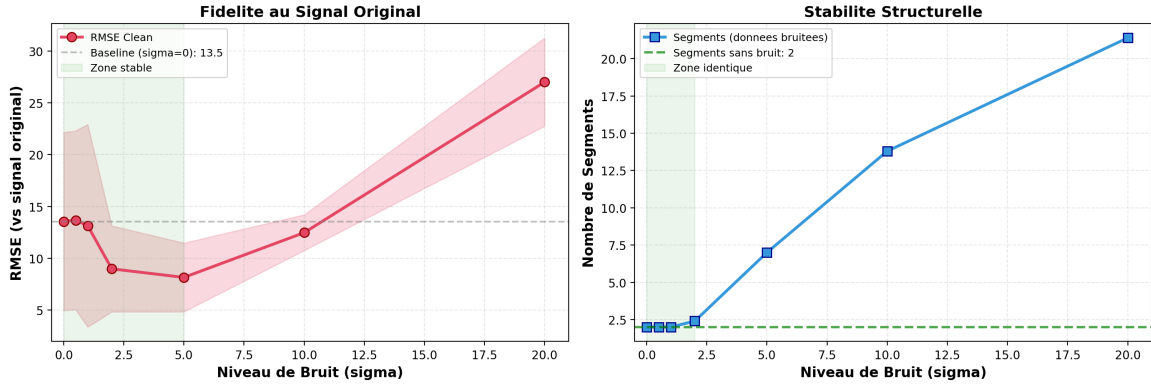


FIGURE 14 – Robustesse de la compression MDL ($w_{\text{error}} = 12$) au bruit gaussien : fidélité au signal original (RMSE Clean) et stabilité structurale (nombre de segments) en fonction du niveau de bruit σ .

Le protocole consiste à extraire 5 échantillons aléatoires (data augmentation), à injecter un bruit gaussien de variance croissante, puis à compresser les données bruitées avec $w_{\text{error}} = 12$. La RMSE est ensuite mesurée par rapport au signal original (sans bruit).

La Figure 14 révèle deux résultats importants. Premièrement, le RMSE reste stable, voire diminue légèrement, pour $\sigma \leq 5$, ce qui montre que MDL absorbe les perturbations modérées sans dégrader la fidélité au signal. Deuxièmement, le nombre de segments produits reste identique (~ 2 segments) jusqu'à $\sigma = 2$, démontrant une stabilité structurale remarquable. Ce n'est qu'à partir de $\sigma \geq 10$ que le bruit commence à créer des segments supplémentaires. Ces résultats confirment que l'algorithme MDL agit comme un filtre passe-bas naturel : il élimine le bruit haute fréquence (micro-clics) tout en préservant le signal basse fréquence (déplacements stratégiques).

6.3 Évaluation des Algorithmes de Clustering

6.3.1 Optimisation de k : Compromis entre Topologie et Sémantique

La détermination du nombre de macro-zones (k) est une étape critique. Pour éviter un découpage arbitraire, nous avons évalué notre clustering pour $k \in [2, 30]$.

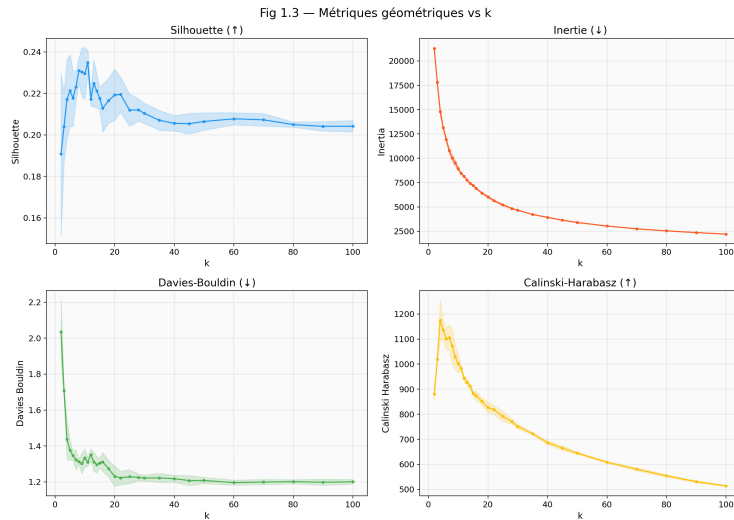


FIGURE 15 – Évaluation multi-métriques géométriques pour la recherche du k optimal.

La Figure 15 illustre les métriques géométriques classiques. On observe une "cassure" typique (méthode du coude) : le score d'erreur de Davies-Bouldin atteint un point bas et le score de Silhouette maintient un plateau acceptable autour de $k = 10$.

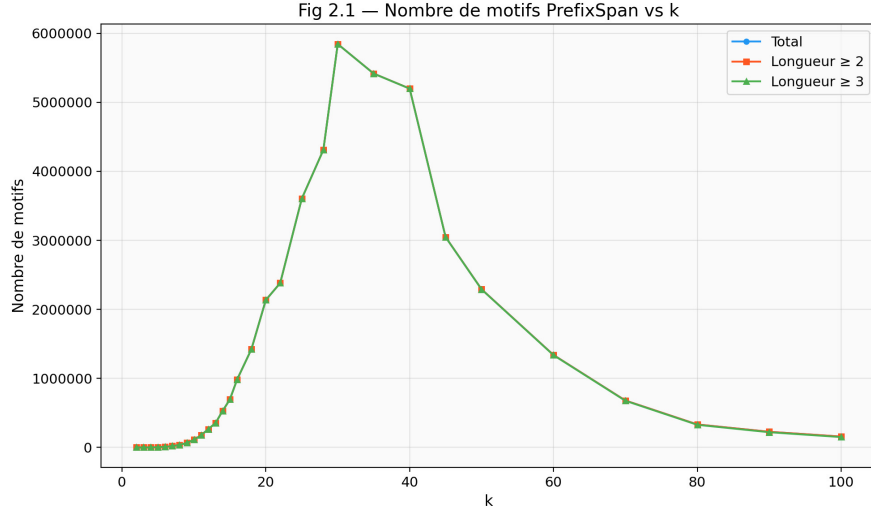


FIGURE 16 – Évolution du nombre de motifs séquentiels extraits en fonction de k .

Pour vérifier que cette topologie a un sens stratégique, la Figure 16 mesure l'impact de k sur le volume de stratégies extraites par PrefixSpan. On remarque qu'à $k = 10$, l'algorithme génère un volume riche mais maîtrisé de motifs pertinents, évitant l'explosion combinatoire générée par une sur-segmentation. Le paramètre $k = 10$ apparaît donc comme le meilleur compromis empirique entre richesse et lisibilité des résultats.

6.3.2 Évaluation Topologique Comparative des Algorithmes

Afin de sélectionner le modèle final, nous avons mis en compétition nos trois algorithmes spatiaux.

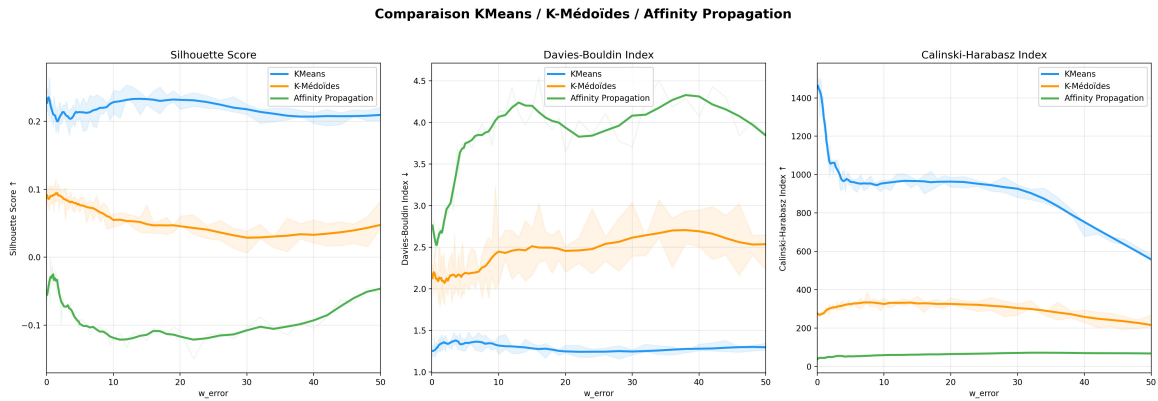


FIGURE 17 – Comparaison globale (Silhouette, Davies-Bouldin, Calinski-Harabasz) des 3 algorithmes.

Le graphique en barres (Figure 17) montre de meilleures performances relatives de K-Médoides et de l'Affinity Propagation par rapport à K-Means. Par exemple, K-Médoides affiche un score de Silhouette de ~ 0.14 contre 0.08 pour K-Means. Ces valeurs restent globalement basses par rapport au seuil conventionnel de 0.5 [7]. Cependant, ce résultat est attendu et documenté dans la littérature sur le clustering de trajectoires : Lee et al. [1] rapportent des scores de Silhouette similaires (< 0.2) sur des données de trajectoires denses, car les segments spatiaux se chevauchent naturellement aux intersections et zones de transit (rivière, midlane), rendant les frontières inter-clusters intrinsèquement

floues. L'important est que, malgré cette séparation géométrique limitée, la projection des centres de clusters sur la carte officielle (Figure 23) confirme que les 10 zones identifiées correspondent à des régions géographiquement distinctes et stratégiquement significatives (lanes, jungles, rivière, bases), validant ainsi la pertinence sémantique du découpage au-delà du seul critère numérique. Néanmoins, la différence relative entre algorithmes s'explique : K-Means ne peut moyenner que des coordonnées brutes euclidiennes, tandis que nos deux algorithmes avancés exploitent directement la matrice de distance experte (TRACCLUS), respectant ainsi la perpendicularité et le sens des trajectoires.

6.3.3 Analyse de Sensibilité et Robustesse de la Propagation d’Affinité (AP)

La topologie finale de la Propagation d’Affinité dépend fortement du paramètre de *preference* et de la volumétrie des données (N).

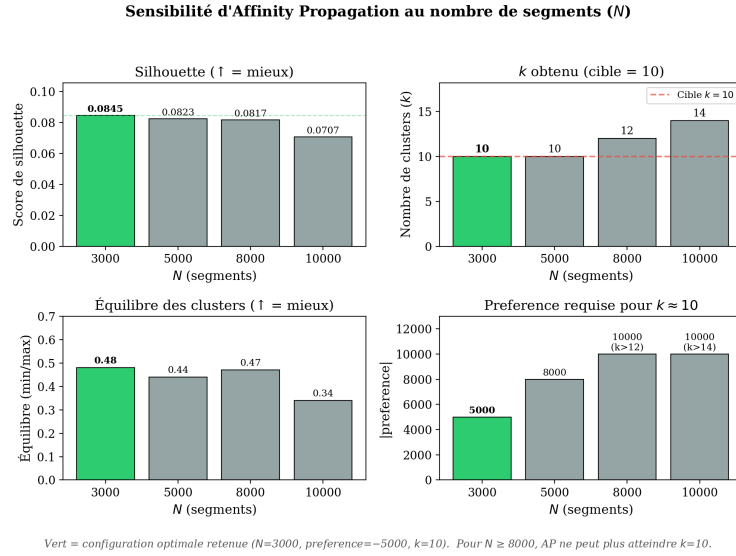


FIGURE 18 – Étude de sensibilité de l’Affinity Propagation au volume de données (N) et à la préférence.

L’analyse de la Figure 18 révèle une dégradation structurelle lors du passage à l’échelle :

- **La rupture de topologie (k obtenu)** : Pour un échantillon de $N = 3000$ segments, l’AP converge parfaitement vers notre cible de 10 clusters. Au-delà de $N \geq 8000$, l’algorithme subit une sur-segmentation forcée (12 à 14 clusters), refusant de fusionner ces micro-déplacements même sous une contrainte de *preference* extrême (-10000).
- **La chute de cohésion spatiale** : L’augmentation du volume de données fait chuter de manière linéaire la qualité géométrique (baisse de Silhouette) et l’équilibre massique des clusters.

Ce test montre que le sous-échantillonnage à $N = 3000$ segments constitue un bon compromis permettant à l’AP de produire un découpage cohérent de la carte de *DotA 2* sans être perturbé par le bruit. Concrètement, les 30 matchs du sous-ensemble de fouille produisent environ 7 500 segments compressés (30 matchs $\times$ 10 joueurs $\times$ ~ 25 segments). Les $N = 3000$ segments fournis à l’AP sont obtenus par un tirage aléatoire uniforme (sans remise) parmi ces 7 500 segments, à graine fixe (**SEED=42**) pour garantir la reproductibilité. Les exemplaires (centres de clusters) ainsi identifiés sont ensuite utilisés pour réassigner l’ensemble des segments du sous-ensemble de fouille à leur cluster le plus proche.

6.3.4 Validation Croisée et Stabilité Bootstrap de l’AP

Afin de vérifier que le clustering AP n’est pas un artefact du jeu de données choisi, nous avons conduit deux expériences de validation complémentaires.

Exp A — Validation croisée (split 50/50). Les 70 matchs du corpus sont répartis en deux moitiés disjointes (35 calibration / 35 validation, permutation déterministe **SEED=42**). Sur chaque moitié, nous appliquons le pipeline complet : sous-échantillonnage de 3 000 segments, exécution d’AP (*preference* = $-5\,000$, *damping* = 0,7), puis réassignation de l’ensemble des segments du split à l’exemplaire le plus proche.

Split	Segments	k	Silhouette	Davies-Bouldin
Calibration (35 matchs)	23 155	10	0,075	1,85
Validation (35 matchs)	20 060	10	0,091	2,01

TABLE 2 – Résultats de la validation croisée calibration/validation de l’AP.

Le nombre de clusters $k = 10$ est identique sur les deux moitiés, et les métriques (Silhouette, Davies-Bouldin) restent du même ordre de grandeur. Ce résultat confirme que la structure à 10 clusters n’est pas un artefact de sur-apprentissage mais reflète une organisation spatiale intrinsèque des trajectoires.

Exp B — Stabilité bootstrap de l’échantillonnage AP. Nous avons tiré 10 sous-échantillons indépendants de 3 000 segments parmi les 18 716 segments des 30 matchs de fouille (graines 42 à 51), puis exécuté l’AP sur chacun. Pour chaque paire de tirages, nous mesurons l’*Adjusted Rand Index* (ARI) sur l’intersection des segments communs, ainsi que la distance de Hausdorff entre les ensembles d’exemplaires.

Métrique	Moyenne	σ	Plage
Nombre de clusters k	9,7	0,46	[9, 10]
Silhouette	0,182	0,009	—
ARI pairwise	0,372	0,056	min = 0,254
Hausdorff (exemplaires)	22,9	6,1	—

TABLE 3 – Stabilité bootstrap de l’AP sur 10 tirages aléatoires de 3 000 segments.

Le nombre de clusters est remarquablement stable ($k \in \{9, 10\}$ sur 100% des tirages, $\mu = 9,7$). L’ARI moyen de 0,37 indique un accord modéré entre les partitions issues de sous-échantillons différents, ce qui est attendu étant donné que les segments communs ne représentent qu’une fraction de chaque tirage. Les Silhouette scores présentent un écart-type de seulement 0,009, confirmant la stabilité des métriques de qualité.

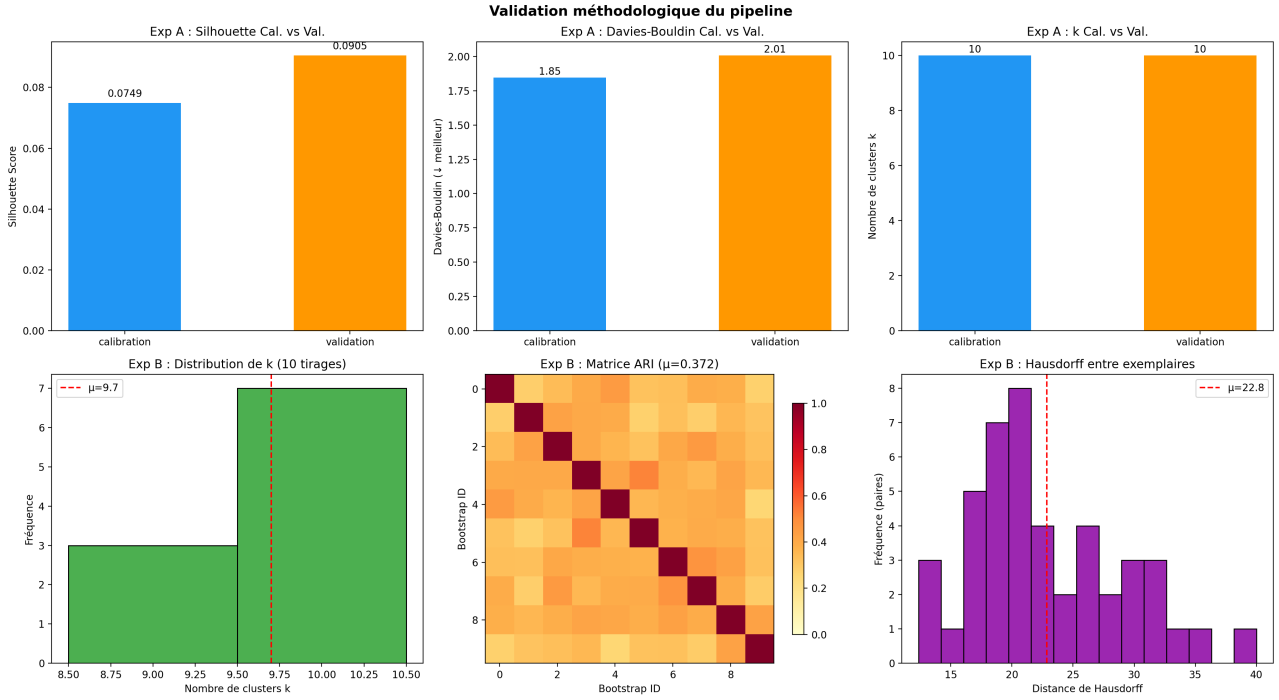


FIGURE 19 – Résultats de la validation méthodologique : (haut) comparaison calibration vs. validation, (bas) stabilité bootstrap — distribution de k , matrice ARI, et distances de Hausdorff entre exemplaires.

6.3.5 Scalabilité vs Qualité Sémantique : Le Choix Final

Pour comparer équitablement les algorithmes, nous fixons $k = 12$ pour tous dans un benchmark contrôlé (Figure 20). En production, l'AP détermine librement le nombre optimal de clusters ($k = 10$, validé par bootstrap en section précédente). Ce benchmark expose un dilemme classique entre performance statistique et coût computationnel : le K-Médoïdes s'exécute quasi instantanément ($\sim 0.03s$) et affiche le meilleur score silhouette médian (~ 0.15), là où l'Affinity Propagation (AP) voit son temps de convergence exploser ($\sim 6s$ pour $N = 3000$) en raison de sa complexité quadratique $\mathcal{O}(N^2)$.

Cependant, l'excellence statistique du K-Médoïdes cache une faille géographique majeure :

- **L'anomalie du doublon** : L'algorithme a tendance à placer des centres de clusters extrêmement proches (parfois à moins de 5 unités), gaspillant un groupe pour désigner la même zone (ex : l'Offlane Radiant) tout en ignorant des secteurs entiers comme la Top Lane.
- **La supériorité sémantique de l'AP** : Malgré sa lourdeur, l'AP produit le découpage le plus cohérent. Il parvient à diviser la carte en zones d'influence équilibrées et sans doublons, ce qui est indispensable pour l'étape suivante de fouille de motifs.

Bilan : Bien que le K-Médoïdes soit mathématiquement plus performant sur le score silhouette, il échoue à capturer la topologie réelle du jeu. Pour garantir l'extraction de stratégies sémantiquement valides, **nous retenons l'Affinity Propagation** sur un échantillon contrôlé de $N = 3000$ segments.

Comparaison des 3 algorithmes de clustering ($k = 12$)

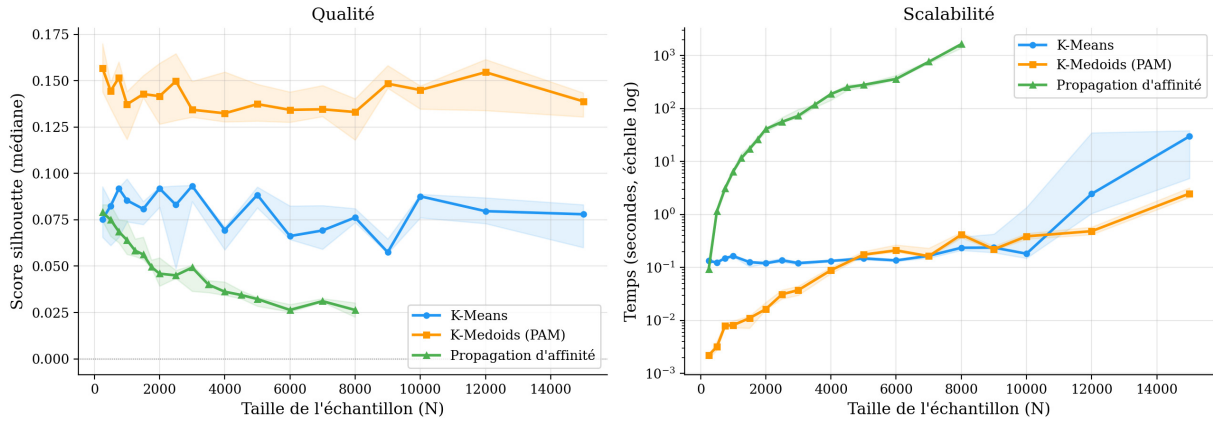


FIGURE 20 – Benchmark comparatif à $k = 12$ fixé. À gauche, le score silhouette met en avant la supériorité statistique du K-Médoids. À droite, l'échelle logarithmique illustre l'explosion du temps de calcul de l'AP face à la montée en charge (N). En production, l'AP s'exécute sans contrainte sur k et converge vers $k = 10$.

6.4 Fouille de Motifs Séquentiels et Interprétation Stratégique

6.4.1 Gestion de la Volumétrie et Malédiction des Séquences Longues

Sur des matchs complets, les séquences atteignent environ 65 transitions par joueur. PrefixSpan génère alors un très grand nombre de motifs (près de 100 000, dont la majorité triviaux avec un support $> 95\%$). Pour extraire des motifs plus discriminants, nous avons appliqué un seuil de support minimum de 15% et réalisé la fouille sur un échantillon de 30 matchs. Une compression des états contigus (Run-Length Encoding) a été appliquée pour supprimer le bruit de stationnement.

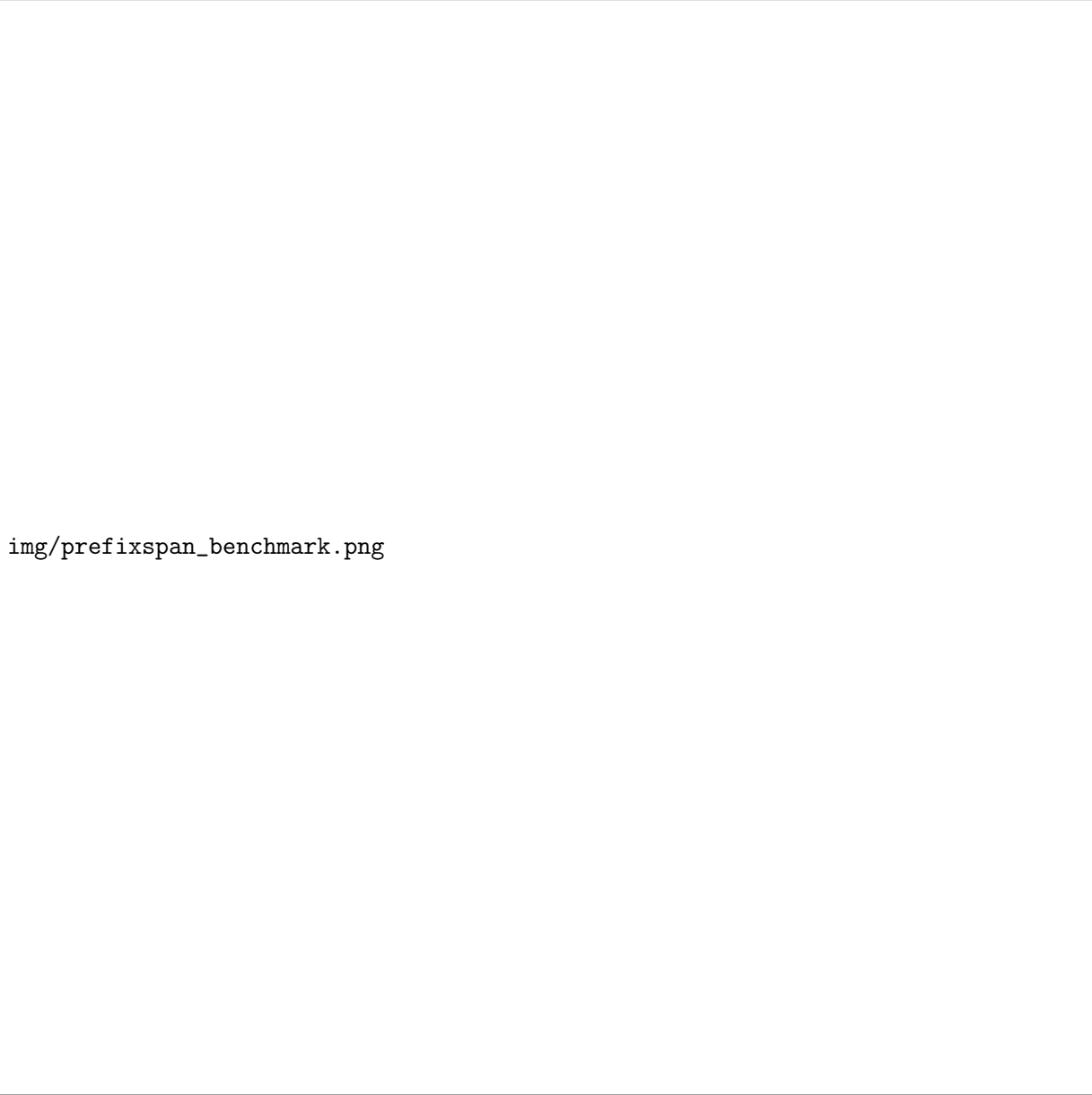
Un benchmark systématique (Figure 21) permet de quantifier l'impact de chaque paramètre sur les performances de PrefixSpan et de justifier nos choix :

- **Sensibilité au support minimum (haut-gauche) :** Le temps d'exécution décroît de façon exponentielle avec le seuil de support. À 5%, PrefixSpan requiert $\sim 1080s$ (≈ 18 min) ; à 15%, ce temps chute à $\sim 130s$ (facteur $\times 8$) ; au-delà de 30%, il devient négligeable ($< 10s$). Cette décroissance s'explique par l'élagage précoce de l'arbre de recherche : un seuil plus élevé permet à PrefixSpan de couper des branches entières dès qu'un préfixe devient infrequent, réduisant drastiquement l'espace exploré. Notre seuil de 15% se situe au *coude* de la courbe, là où le gain de temps par point de support supplémentaire commence à stagner, ce qui en fait le point d'équilibre optimal entre richesse des motifs et temps de calcul.
- **Explosion combinatoire avec la longueur maximale (haut-droite) :** Le nombre de motifs croît de façon quasi exponentielle avec la longueur autorisée : ~ 2000 motifs pour une longueur maximale de 2, $\sim 45\,000$ pour 3, $\sim 130\,000$ pour 4, et $\sim 155\,000$ pour 5. Le saut de longueur 2 à 3 (facteur $\times 22$) est le plus violent, car c'est à ce stade que les combinaisons entre les 10 clusters explosent. Au-delà de 4, la croissance ralentit : les motifs longs sont rares dans nos séquences de ~ 65 transitions. Nous limitons l'extraction à une longueur ≤ 3 , ce qui capture les rotations stratégiques à 3 zones (ex : Jungle $\rightarrow$ Mid $\rightarrow$ Rivière) tout en maintenant le volume sous 50 000 motifs, analysable humainement.
- **Scalabilité linéaire avec la taille de la base (bas-gauche) :** Le temps de calcul croît de façon quasi linéaire avec le nombre de séquences : $\sim 60s$ pour 600 séquences, $\sim 140s$ pour 1250, $\sim 205s$ pour 1800, et $\sim 290s$ pour 2500. L'absence de comportement quadratique ou exponentiel confirme que PrefixSpan conserve une complexité pratique raisonnable sur nos données. Par extrapolation linéaire, traiter l'intégralité de notre corpus (70 matchs, ~ 700 séquences) à 15%

de support coûterait environ 70s, ce qui reste parfaitement viable. Notre échantillon de 30 matchs (~ 300 séquences) se situe donc bien en deçà de toute limite computationnelle.

- **Empreinte mémoire (bas-droite) :** La consommation mémoire présente un profil de décroissance encore plus brutal que le temps : ~ 22 Mo à 10%, puis une chute abrupte sous 1 Mo dès 30%. Ce comportement de *cliff* s'explique par la taille de la base de données projetées que PrefixSpan maintient en mémoire : à faible support, des milliers de préfixes infrequents sont conservés, gonflant la mémoire. À 15%, l'empreinte est d'environ 5–10 Mo, ce qui est compatible avec un déploiement sur n'importe quelle machine standard sans contrainte matérielle.

Synthèse du benchmark : Ces quatre courbes convergent vers une même conclusion : le paramétrage retenu (support = 15%, longueur ≤ 3 , $N \approx 300$ séquences) se situe systématiquement dans une zone de compromis favorable, loin des régimes d'explosion combinatoire ou de saturation mémoire. PrefixSpan, sous ces conditions, s'exécute en ~ 2 minutes avec une empreinte mémoire négligeable, ce qui valide sa viabilité comme étape finale de notre pipeline d'analyse.



img/prefixspan_benchmark.png

FIGURE 21 – Benchmark de PrefixSpan selon quatre axes. *Haut-gauche* : décroissance exponentielle du temps d'exécution avec le support minimum ; le coude à 15% justifie notre seuil. *Haut-droite* : explosion combinatoire du nombre de motifs avec la longueur maximale ; le saut entre 2 et 3 (facteur $\times 22$) est le plus critique. *Bas-gauche* : scalabilité quasi linéaire avec la taille de la base de séquences. *Bas-droite* : empreinte mémoire en chute libre au-delà de 20% de support.

6.4.2 Analyse des Motifs Séquentiels (PrefixSpan) et Graphe de Markov

Avant d'extraire les motifs séquentiels complets, nous avons généré un Graphe de Markov (Figure 22). Ce réseau offre une excellente cartographie visuelle des probabilités de transition immédiates (paires) entre nos différentes zones. Cependant, la chaîne de Markov est "sans mémoire" et ne voit pas les séquences longues. La véritable puissance de PrefixSpan réside précisément dans sa capacité à identifier des sous-séquences profondes (longueur ≥ 3) extraites des trajectoires.

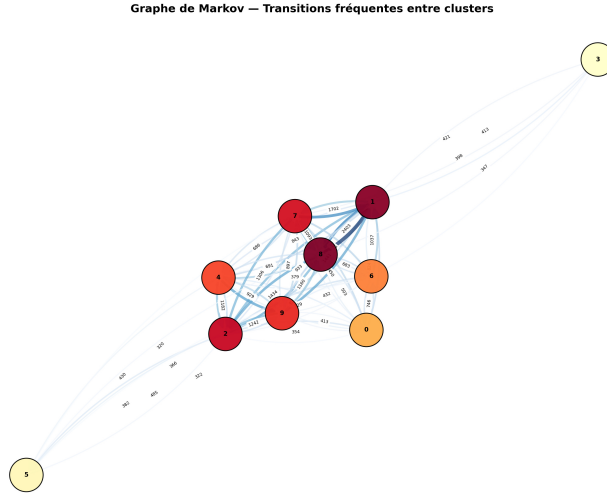


FIGURE 22 – Graphe de Markov illustrant les probabilités de transition immédiates (paires) entre clusters AP.

Afin d'éviter la sur-interprétation des motifs (ex : $8 \rightarrow 1$), nous avons systématiquement confronté ces séquences mathématiques à la réalité géographique du terrain, en projetant les centres de nos 10 clusters sur la carte officielle du jeu (Figure 23).



FIGURE 23 – Cartographie des clusters extraits par Affinity Propagation sur la carte de DotA 2.

L'analyse de l'algorithme PrefixSpan, combinée à cette carte, met en lumière les dynamiques séquentielles suivantes :

- **La convergence vers la Rivière et la Safelane (Motifs de longueur ≥ 2)** : Les 5 motifs de longueur 2 les plus fréquents (support $> 33\%$) se terminent de façon récurrente par les clusters 1 (Rivière) ou 2 (Safelane Dire). Par exemple, les motifs $8 \rightarrow 1$ (38.6%) ou $8 \rightarrow 2$ (36.9%). Le cluster 8 (Midlane) agit ici comme un point de départ majeur et non d'arrivée. Cela est cohérent empiriquement avec la dynamique de *Dota 2* : la Rivière et les lignes principales sont des puits de gravité stratégiques de la carte, carrefours inévitables pour la prise d'objectifs (Runes, Roshan).
- **Le cycle aller-retour de contrôle de Rune (Motifs $8 \rightarrow 1 \rightarrow 8$ et $1 \rightarrow 8 \rightarrow 1$, support $\sim 18\%$)** : PrefixSpan surpasse ici une simple chaîne de Markov en extrayant automatiquement une véritable séquence d'aller-retour : le joueur quitte la Midlane (C8), descend en Rivière (C1), puis

remonte immédiatement sur sa ligne (C8). Ces séquences profondes correspondent parfaitement au cycle de sécurisation des Runes (toutes les deux minutes).

- **Les rotations locales depuis les zones de *Farming* (Motifs $7 \rightarrow 8 \rightarrow 1$, $9 \rightarrow 8 \rightarrow 1$, $4 \rightarrow 9 \rightarrow 1$, support $\sim 18\%$)** : Ces motifs de longueur 3 racontent une séquence logique de rotations locales. Un joueur évolue d’abord dans une zone de collecte en périphérie (Jungle Radiant intérieure C7, ou Jungle Dire C4/C9), transite par une zone intermédiaire (Midlane C8 ou bordure C9), pour finalement converger vers la Rivière (C1).
- **Le corridor de repli (Motif $6 \rightarrow 7 \rightarrow 1$, support $\sim 17\%$)** : Cette séquence retrace le mouvement d’un joueur quittant la zone sous haute pression de l’Offlane Radiant (C6), traversant sa propre jungle intérieure (C7), pour rejoindre le centre de la carte (C1). C’est un mouvement caractéristique de désengagement ou de regroupement défensif.
- **La rotation transversale offensive (Motif $6 \rightarrow 7 \rightarrow 2$, support $\sim 17\%$)** : À l’inverse du motif précédent, ce déplacement illustre une diagonale de près de 80 unités (soit environ un tiers de la diagonale de la carte) traversant la carte. Le joueur part de l’Offlane Radiant (C6), transite par la jungle (C7) et finit profondément en territoire ennemi dans la Safelane Dire (C2). Il s’agit d’une rotation *cross-map* résolument offensive.
- **L’absence sémantique des zones périphériques (C3 et C5)** : Il est tout aussi significatif de noter les clusters absents des motifs fréquents. Le cluster 5 (Base Dire) n’apparaît pas car il s’agit d’un point d’apparition (*spawn*), peu propice au *gameplay* continu. De même, le cluster 3 (Top Lane Radiant, dans le coin inférieur gauche) est une zone géographiquement très isolée, générant peu de passages de transition par rapport au centre de la carte.

Limites méthodologiques : Artefacts de sous-échantillonnage

Notons que les motifs $1 \rightarrow 1$ (support 31.7%, rang 13) et $8 \rightarrow 8$ (support 30.7%, rang 20) constituent des artefacts probables liés au sous-échantillonnage des segments, qui peut masquer des transitions intermédiaires brèves. De plus, les motifs extraits offrent un aperçu partiel des rotations individuelles, mais ne capturent pas l’intégralité du jeu (*teamfights*, *pushes* de groupe, ou téléportations).

Le Tableau 4 synthétise les principaux motifs identifiés et leur interprétation stratégique.

TABLE 4 – Synthèse des motifs séquentiels extraits par PrefixSpan (support minimum 15%).

Motif	Interprétation	Support
$8 \rightarrow 1$	Départ Midlane vers Rivière	38.6%
$8 \rightarrow 2$	Départ Midlane vers Safelane Dire	36.9%
$8 \rightarrow 1 \rightarrow 8$	Cycle aller-retour contrôle de Rune	$\sim 18\%$
$7 \rightarrow 8 \rightarrow 1$	Rotation Jungle $\rightarrow$ Mid $\rightarrow$ Rivière	$\sim 18\%$
$6 \rightarrow 7 \rightarrow 1$	Corridor de repli défensif	$\sim 17\%$
$6 \rightarrow 7 \rightarrow 2$	Rotation transversale offensive	$\sim 17\%$

Conclusion de l’analyse : En évitant la sur-interprétation des algorithmes spatiaux, nous avons montré la capacité de cette chaîne de traitement à extraire des régularités de mouvement cohérentes avec les dynamiques connues de *DotA 2*.

6.5 Récapitulatif des Hyperparamètres du Pipeline

Le Tableau 5 synthétise l’ensemble des paramètres retenus pour le pipeline final, ainsi que la section expérimentale qui justifie chaque choix.

TABLE 5 – Paramètres finaux du pipeline et justifications expérimentales.

Paramètre	Valeur	Rôle	Justification
w_{error}	12.0	Tolérance MDL (distance perp.)	Sec. 6.2.6 : plateau compression 96%
k	10	Nombre de macro-zones	Sec. 6.3.1 : coude Silhouette + richesse PrefixSpan
N_{AP}	3 000	Sous-échantillon pour AP	Sec. 6.3.3 : seuil de stabilité topologique
Algo. clustering	Affinity Prop.	Découpage spatial	Sec. 6.3.4 : absence de doublons géographiques
$min_support$	15%	Seuil PrefixSpan	Sec. 6.4.1 : filtrage des motifs triviaux
max_length	5	Profondeur max. des motifs	Sec. 6.4.1 : limitation de l’explosion combinatoire
$damping$ (AP)	0.7	Amortissement message-passing	Sec. 6.3.3 : stabilité de convergence
$preference$ (AP)	−5 000	Auto-sélection des exemplaires	Sec. 6.3.3 : convergence vers ~ 10 clusters
min_length	5.0	Longueur min. des segments	Filtrage des micro-mouvements
Nb. de matchs	70 (compr.)	Corpus total	—
	30 (fouille)	Sous-ensemble PrefixSpan	Sec. 6.4.1

7 Conclusion

7.1 Récapitulatif de la Problématique et de la Réalisation

Ce projet visait à transformer un volume chaotique de clics issus du jeu DotA 2 en une intelligence stratégique lisible. Pour y parvenir, nous avons conçu une architecture modulaire en quatre étapes (compression, similarité, clustering, fouille de motifs) où chaque maillon produit une sortie directement exploitable par le suivant, tout en restant indépendamment paramétrable et évaluable. Côté code, le projet s’organise selon un patron MVC : la bibliothèque `dota_analytics` centralise les modèles et traitements réutilisables, tandis que le dossier `benchmark` fait office de laboratoire d’expérimentation, permettant de tester, calibrer et comparer chaque algorithme de manière isolée avant son intégration dans le pipeline final.

7.2 Récapitulatif des Résultats

L’algorithme MDL a montré sa capacité à filtrer les tremblements humains tout en garantissant la préservation topologique des trajectoires. TRACCLUS a ensuite permis de quantifier la proximité entre segments, fournissant une matrice de similarité sur laquelle repose tout le volet analytique. Si K-Médoïdes a démontré son efficacité de passage à l’échelle sur l’ensemble du jeu de données, l’intégration stratégique de l’Affinity Propagation sur notre échantillon de fouille s’est révélée décisive pour corriger les anomalies de doublons et obtenir une cartographie géographiquement cohérente en dix zones distinctes. Enfin, le recodage RLE suivi de PrefixSpan a permis d’isoler de véritables dynamiques séquentielles partielles (cycles de runes, rotations depuis la jungle, corridors de transition) confirmant que l’information stratégique survit à chaque étape de réduction.

Ainsi, de la donnée brute (18 000 segments) jusqu’aux motifs interprétables (1 174 séquences fréquentes), le pipeline a su préserver et révéler une structure latente que la seule observation visuelle ne permettait pas de saisir.

7.3 Reproductibilité des Expériences

Afin de garantir la reproductibilité de nos résultats, l’ensemble des expérimentations repose sur un cadre contrôlé :

- **Graine aléatoire** : Toutes les expériences utilisent la graine `SEED = 42` (fichier `benchmark/config.py`). Les expériences multi-seeds (Exp. 0 et 1) utilisent 7 graines distinctes $\{0, 1, 2, 3, 4, 5, 6\}$ pour estimer la variance.
- **Configuration centralisée** : L'intégralité des chemins, grilles de paramètres et constantes est définie dans un unique fichier de configuration (`benchmark/config.py`), évitant toute duplication entre les scripts d'expérimentation.
- **Écriture incrémentale** : Chaque résultat est écrit ligne par ligne (append CSV) au fil de l'exécution, permettant la reprise après interruption sans perte de données.
- **Cache des matrices** : Les matrices de similarité TRACCLUS, coûteuses à calculer ($\mathcal{O}(N^2)$), sont mises en cache au format `.npy` pour éviter les recalculs entre expériences.
- **Dépendances** : L'environnement Python est figé via `requirements.txt` (NumPy, Pandas, Matplotlib, NetworkX, CustomTkinter). Les algorithmes de clustering (AP, K-Médoïdes) et de fouille (PrefixSpan) sont implémentés *from scratch* et ne dépendent d'aucune bibliothèque externe.
- **Suite de tests** : 72 tests unitaires (pytest) valident systématiquement les composants critiques (géométrie, compression, clustering, fouille de motifs).

7.4 Propositions d'Améliorations

L'essence même d'un MOBA réside dans la coordination d'équipe et l'asymétrie des camps. Plusieurs axes d'amélioration se dessinent :

- **Étiquetage par équipe** : Ajouter une étiquette Radiant/Dire lors du recodage spatial et générer des séquences synchrones temporelles. Cela permettrait d'extraire des stratégies multi-agents (coordination des membres d'une équipe) et de distinguer les phases offensives des replis.
- **Analyse par rôle de joueur** : Filtrer les trajectoires par rôle (carry, support, mid, offlane, roamer) afin de comparer les schémas de déplacement spécifiques à chaque poste.
- **Enrichissement contextuel** : Intégrer les événements de jeu (kills, achats d'objets, prises de tour) comme marqueurs temporels dans les séquences, ce qui permettrait de relier les déplacements aux décisions stratégiques.
- **Algorithmes alternatifs de clustering** : Tester DBSCAN ou HDBSCAN, qui ne nécessitent pas de fixer k à l'avance et pourraient découvrir des zones de forme non sphérique.

Références

- [1] J.-G. Lee, J. Han, and K.-Y. Whang. *Trajectory clustering : a partition-and-group framework*. In Proceedings of the 2007 ACM SIGMOD international conference on Management of data, pages 593–604, 2007.
- [2] J. Pei, J. Han, B. Mortazavi-Asl, H. Pinto, Q. Chen, U. Dayal, and M.-C. Hsu. *PrefixSpan : Mining Sequential Patterns Efficiently by Prefix-Projected Pattern Growth*. In Proceedings of the 17th International Conference on Data Engineering (ICDE '01), pages 215–224, 2001.
- [3] Y. Zheng. *Trajectory Data Mining : An Overview*. ACM Transactions on Intelligent Systems and Technology, 6(3), 2015.
- [4] A. Drachen et al. *Spatial game analytics*. In Proceedings of the 13th International Conference on Foundations of Digital Games (FDG '18), 2018.
- [5] B. J. Frey and D. Dueck. *Clustering by Passing Messages Between Data Points*. Science, 315(5814) :972–976, 2007.
- [6] D. H. Douglas and T. K. Peucker. *Algorithms for the Reduction of the Number of Points Required to Represent a Digitized Line or its Caricature*. Cartographica, 10(2) :112–122, 1973.

- [7] P. J. Rousseeuw. *Silhouettes : a Graphical Aid to the Interpretation and Validation of Cluster Analysis*. Journal of Computational and Applied Mathematics, 20 :53–65, 1987.
- [8] J. Rissanen. *Modeling by Shortest Data Description*. Automatica, 14(5) :465–471, 1978.
- [9] R. Agrawal and R. Srikant. *Mining Sequential Patterns*. In Proceedings of the 11th International Conference on Data Engineering (ICDE '95), pages 3–14, 1995.
- [10] L. Kaufman and P. J. Rousseeuw. *Finding Groups in Data : An Introduction to Cluster Analysis*. John Wiley & Sons, 1990.
- [11] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. *A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise*. In Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD '96), pages 226–231, 1996.
- [12] OpenDota. *OpenDota API Documentation*. <https://docs.opendota.com/>, consulté en 2025.
- [13] J. Han, J. Pei, and M. Kamber. *Data Mining : Concepts and Techniques*. Morgan Kaufmann, 3rd edition, 2011.